OnChainAgent: LLM-Powered Multi-Agent Systems for Autonomous Operations on the Ethereum Mainnet

Jiashun Wang
Shanghai University of International Business and Economics
Shanghai, China
24089054@suibe.edu.cn

Zhi Yang
Shanghai University of Finance and Economics
Shanghai, China
yangzhi@stu.sufe.edu.cn

Guangyan Gan
Nanyang Technological University
Singapore, Singapore
GUANGYAN001@e.ntu.edu.sg

Hanzhang Qin*
National University of Singapore
Singapore, Singapore
hzqin@nus.edu.sg

Liwen Zhang
Shanghai University of Finance and Economics
Shanghai, China
zhang.liwen@shufe.edu.cn

Lin William Cong
Nanyang Technological University
Singapore, Singapore
will.cong@ntu.edu.sg

Abstract

Operations on the Ethereum mainnet expose powerful financial and ownership primitives, but they also couple user requests to rapidly changing wallet state, protocol conditions, and irreversible side effects. As a result, natural-language automation for Ethereum-mainnet operations is not only an intent-understanding problem; it is a state-aware execution-control problem. We present **OnChainAgent**, an LLM-powered multi-agent system for autonomous operations on the Ethereum mainnet. OnChainAgent organizes the workflow into a Planner, specialized Tool Agents, a Reviewer, and a Decision Agent that operate over shared blockchain state in our current Ethereum-mainnet setting. This design targets a specific systems objective: given a user request, the system should select the correct execution outcome among `query`, `executed`, `quote_only`, and `blocked`, rather than merely maximize task completion. We evaluate OnChainAgent in a simulated Ethereum-mainnet execution environment on 84 benchmark cases and compare it against a Web3Agent-style single-agent baseline and a no-reviewer ablation. OnChainAgent achieves the strongest overall Case Pass Rate, with its clearest gains on safety-sensitive quote-only and blocked/intervention cases. These results suggest that role-specialized LLM agents, coupled through shared state and explicit review, provide a practical abstraction for dependable language-driven interaction with the Ethereum mainnet.

Keywords

Blockchain Systems, On-chain Operations, Large Language Models, Multi-agent Systems

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
NBC 2026, XXX, XXX
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Jiashun Wang, Zhi Yang, Guangyan Gan, Hanzhang Qin, Liwen Zhang, and Lin William Cong . 2026. OnChainAgent: LLM-Powered Multi-Agent Systems for Autonomous Operations on the Ethereum Mainnet. In *Proceedings of Nanyang Blockchain Conference (NBC 2026).* ACM, New York, NY, USA, 8 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The Ethereum mainnet has become a primary programmable settlement layer for digital assets, smart-contract execution, and composable financial activity [19]. Yet operating directly on it remains difficult for ordinary users. Even common actions such as managing ETH or ERC-20 balances, approving a token, calling a contract, swapping assets, or preparing a bridge from Ethereum mainnet context require users to navigate fragmented interfaces, protocol-specific procedures, and substantial execution risk [3, 29]. A useful agent for Ethereum-mainnet operations must therefore do more than understand intent: it must remain grounded in current wallet and protocol state and choose the correct execution level under risk.

Recent advances in large language models suggest that agents could serve as a natural-language control layer for Ethereum mainnet execution [6, 13, 18, 21, 24]. However, this setting is not just another tool-use problem. Requests such as "transfer 0.2 ETH to this address" or "swap 100 USDC for ETH" cannot be resolved from text alone. The system must reason over balances, allowances, gas, quote freshness, route support, and the consequences of broadcasting a transaction [3]. In this setting, the correct response is often not to execute, but to answer a query, return a quote, or block the request. Execution control for Ethereum-mainnet operations should therefore be treated as a blockchain systems problem rather than only a prompt-engineering problem.

Recent systems such as Web3Agent and Eliza establish the feasibility of language-driven blockchain interaction [3, 16]. Our focus is narrower: safety-sensitive operations on the Ethereum mainnet. Here, single-agent pipelines remain structurally limited because they concentrate planning, execution, and judgment in one loop, making it harder to separate tool success from final safety decisions under shared state [1]. This creates a natural case for multi-agent decomposition with clearer role boundaries.

We present **OnChainAgent**, an LLM-powered multi-agent system for autonomous operations on the Ethereum mainnet. OnChainAgent separates the workflow into a *Planner*, specialized *Tool Agents*, a *Reviewer*, and a *Decision Agent*. These roles operate over shared blockchain state in our current Ethereum-mainnet setting so that wallet status, balances, approvals, quotes, and simulation signals remain available throughout the execution loop. Our goal is not only to improve task completion, but to improve *execution-level control*: the system should correctly distinguish among `query`, `executed`, `quote_only`, and `blocked` outcomes under realistic runtime conditions.

In this paper, we make three contributions. First, we introduce a role-specialized multi-agent architecture for Ethereum-mainnet operations that coordinates planning, specialized tool execution, review, and final decision making through shared state. Second, we design an explicit review-centered execution-control mechanism that separates tool success from final safety judgment and treats `query`, `executed`, `quote_only`, and `blocked` as distinct first-class outcomes. Third, we present an 84-case benchmark for autonomous operations on the Ethereum mainnet, covering query, executed, quote-only, and blocked/intervention settings. On this benchmark, OnChainAgent achieves the best overall Case Pass Rate at 79.8%, reaches 100.0% on quote-only cases and 81.0% on blocked/intervention cases, and the matched ablation further shows that the Reviewer is the main source of these safety-control gains.

The rest of this paper is organized as follows. Section 2 reviews related work on autonomous agents, safe tool use, and blockchain execution. Section 3 formalizes the task setting and the decision outcomes considered in this work. Section 4 presents OnChainAgent and its role-specialized multi-agent design. Section 5 introduces the benchmark used in this study. Section 6 then describes the experimental protocol, compared systems, and evaluation results. Finally, Section 7 concludes and discusses future work.

2 Related Work

2.1 AI for Blockchain

Recent work has explored how AI techniques can support blockchain systems at multiple levels, including transaction analysis, smart contract security, and natural-language interfaces for on-chain operations. Earlier studies mainly applied machine learning and deep learning to blockchain data analysis and vulnerability detection, such as phishing account detection on Ethereum transaction graphs and smart contract auditing based on learned representations or hybrid analysis pipelines [7–9, 25–28]. More recent work has begun to use large language models for smart contract auditing and blockchain-facing automation [10, 20, 22]. Among the most relevant systems, Web3Agent [3] shows that language models can translate natural-language instructions into executable on-chain workflows, while Eliza [16] reflects broader interest in deploying AI agents in crypto-native environments. Web3Agent is the closest execution-oriented reference point for our work, but the emphasis is different: it demonstrates modular single-agent orchestration for Web3 task completion, whereas our work centers on execution-level control under shared state. That difference is most visible in how we isolate review and blocked-case judgment rather than only end-to-end task completion.

2.2 LLM-based Agents

Recent LLM-based agent work provides much of the methodological foundation for OnChainAgent. ReAct [24] and Toolformer [13] established influential patterns for combining reasoning and external tool use, while Gorilla [11] and ToolLLM [12] pushed this direction toward larger API collections and more reliable tool invocation. These works show that language models can move beyond static text generation and interact with dynamic software environments through structured actions. At the same time, several systems argue that complex tasks benefit from explicit multi-agent decomposition. HuggingGPT [14] uses an LLM to orchestrate specialized models, while AutoGen [21], MetaGPT [6], and AgentVerse [1] demonstrate that planning, acting, critique, and synthesis can be distributed across role-specialized agents. Critique-oriented methods such as Reflexion [15] further suggest that intermediate reasoning and action traces should be evaluated rather than accepted blindly. These ideas strongly influence our design, especially the decision to separate planning, specialized execution, review, and final decision making. Related work on interactive agents also helps situate our evaluation choices. Benchmarks such as WebVoyager [5] and OSWorld [23] show that agents operating in dynamic environments must handle long-horizon interaction, noisy observations, and cross-step state dependencies. Although these benchmarks are not blockchain-specific, they reinforce the broader point that realistic agent systems should be evaluated as execution systems rather than purely as text generators.

3 Problem Formulation

We study *state-aware natural-language on-chain task solving*. Let x denote a natural-language user request and let s denote the runtime state. In our setting, s may include the currently active wallet, locally available wallets, chain identifiers, token balances, spender allowances, quote results, bridge availability, and safety-related simulation signals. The goal is to transform (x, s) into a final output y that is operationally coherent, state aware, and safety aligned. OnChainAgent represents an intermediate task state as

$$g = (z, \theta, \pi, m),$$

where z is the task type, θ is the grounded parameter set, $\pi = (e_1, \ldots, e_K)$ is an ordered sequence of tool-oriented steps, and m is the execution mode. This formulation is useful because the same user request may legitimately map to different outputs under different runtime states. A swap request may be `quote_only` when the user explicitly asks for simulation, `executed` when all conditions are satisfied and broadcasting is intended, or `blocked` when balance, route, or policy constraints fail.

4 OnChainAgent

4.1 Architecture Overview

OnChainAgent consists of four explicit roles: a Planner, specialized Tool Agents, a Reviewer, and a Decision Agent. The Planner transforms a natural-language request into a structured plan. Tool Agents execute narrow operational families such as wallet management, balance lookup, approval updates, transfers, swaps, and bridges. The Reviewer inspects intermediate results for safety and policy compliance. The Decision Agent synthesizes the trajectory

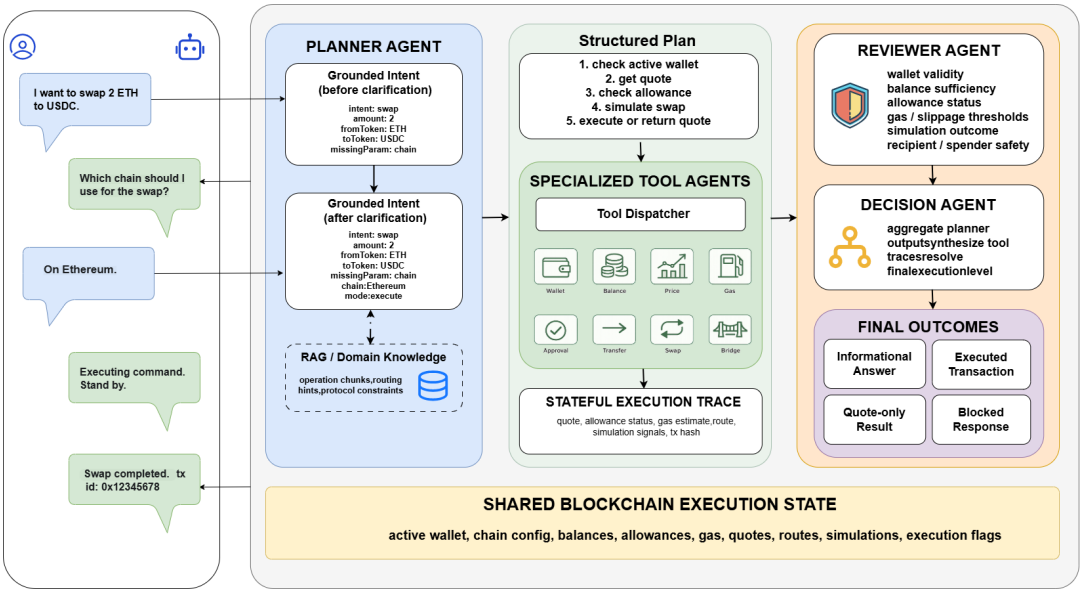

Figure 1: Overview of OnChainAgent. The system consists of a Planner, specialized Tool Agents, a Reviewer, and a Decision Agent.

into the final user-facing response. Figure 1 illustrates the overall architecture and the interactions among these four roles.

This role separation is central rather than cosmetic. Planning, tool selection, safety judgment, and final response generation are all influenced by the same runtime state, but they emphasize different forms of reasoning and different failure modes. Separating them makes the system more inspectable, aligns the architecture with our evaluation metrics, and allows us to isolate which component is responsible for quote-only preservation and blocked/intervention judgment.

4.2 Planner

The Planner handles intent recognition, parameter grounding, and step ordering. Given a request such as "swap 100 USDC to ETH on Base, but only show me the quote," it must identify the task type, recover token and amount parameters, classify the desired execution mode, and determine whether prerequisite steps such as price lookup or allowance inspection are needed. The Planner does not execute chain actions itself; instead, it routes the task into a structured sequence of tool-oriented steps that make the intended execution mode explicit from the beginning.

These steps are then consumed by specialized Tool Agents. The Planner therefore defines the operational intent and dependency structure, while tool execution is delegated to agents whose interfaces and policy checks are tied to concrete on-chain operation families.

4.3 Specialized Tool Agents

Specialized Tool Agents form the execution layer of OnChainAgent. In the runtime trajectory, they are invoked after the Planner produces a structured plan and before the Reviewer validates the resulting trace. The implementation instantiates eight tool agents: `WalletAgent`, `BalanceAgent`, `PriceAgent`, `GasAgent`, `ApprovalAgent`, `TransferAgent`, `SwapAgent`, and `BridgeAgent`. These agents correspond to the tool families `manage_wallet`, `get_balance`, `get_price`, `estimate_gas`, `manage_approval`, `transfer`, `swap`, and `bridge`. The Planner maps each step to one of these tool families, and the system dispatches the step to the corresponding agent.

This specialization is important because different on-chain operations require different state views and different safety conditions. Read-only agents expose wallet, balance, price, gas, allowance, and bridge-status information without broadcasting transactions. Write-capable agents such as approval, transfer, swap, and bridge agents additionally support execution-level controls including dry-run, quote-only mode, simulation, and broadcast gating. Their outputs are structured rather than free-form: a result can report fields such as `executed`, `quote_only`, `blocked`, `simulation_passed`, `warnings`, `violations`, `policy_reason`, and transaction hashes when broadcast occurs.

Tool Agents also update and consume shared state. A wallet operation may change the active account used by later calls; a balance or allowance lookup can determine whether a later transfer or swap is feasible; a bridge quote can expose route support, estimated duration, gas cost, and fallback attempts. In this sense, Tool Agents are not merely API wrappers. They are typed operational adapters that convert planned steps into stateful execution traces. However, their local policy checks are still not a substitute for the Reviewer. Tool-level blocking can prevent some unsafe broadcasts, but the Reviewer is responsible for judging whether the overall trace satisfies the user intent and safety oracle.

This adapter view also defines how OnChainAgent connects to real Web3 infrastructure. The Tool Agents are designed as blockchain-facing execution adapters rather than text-only function callers. In a live Web3 environment, read-only agents can query RPC providers

or indexing services for wallet state, balances, allowances, gas estimates, prices, and bridge availability. Write-capable agents construct typed requests for approvals, transfers, swaps, and bridges, and route them through simulation, policy checks, wallet signing, and broadcast interfaces. For swaps and bridges, the agents can obtain executable quotes and route metadata from DEX aggregators or bridge routers, including slippage, gas, and settlement information. Each backend response is normalized into a structured trace with fields such as chain, token, amount, recipient, spender, quote, simulation result, warnings, signature status, broadcast status, and transaction hash when available.

4.4 Reviewer

The Reviewer is the explicit safety-control component of OnChainAgent. Its role is to decide whether a technically valid intermediate result should be accepted as a valid system outcome. This distinction matters in blockchain systems because tool availability alone is not a sufficient criterion for broadcast. A route can exist while still violating user intent or policy. A transfer can be syntactically valid while still failing balance or recipient safety checks. In our design, the Reviewer is therefore not an auxiliary checker but the component that turns raw execution traces into execution-level judgments.

The Reviewer checks wallet validity, recipient-address safety, amount constraints, approval policy, gas and slippage thresholds, quote freshness, bridge support, and simulation outcomes. It can approve broadcast-ready execution, preserve a `quote_only` outcome, or block the request.

4.5 Decision Agent

The Decision Agent is the final synthesis layer of OnChainAgent. It aggregates the Planner output, Tool Agent traces, and Reviewer verdict into the final user-facing response. Its role is not to invoke additional protocol steps, but to resolve the overall execution state into one coherent outcome. Depending on the case, this outcome may be an informational answer, an executed operation, a quote-only result, or a blocked response.

4.6 Shared State and Execution Policy

All agent roles operate over shared execution state. This state stores wallet activation status, chain configuration, balances, allowances, quote results, simulation metadata, and execution flags. Shared state allows one agent's output to become another agent's operating context and makes multi-step composite execution possible.

This also clarifies the role of execution policy in OnChainAgent. The system is not optimized to maximize broadcast frequency. It is optimized to select the correct execution level: `query`, `executed`, `quote_only`, or `blocked`. That design choice is what makes the Reviewer and Decision Agent meaningful components rather than redundant reasoning hops.

5 Benchmark

Existing public Web3-related benchmarks do not directly fit our evaluation goal, so we construct our own offline benchmark. AI-ABC is a recent crypto-agent benchmark that covers product-facing tasks such as DEX interaction, wallet use, market analysis, project research, and on-chain investigation [4]. However, its cases are represented as structured multi-turn task files with predefined context and scoring rules, rather than as a plain collection of natural-language user requests. This format is useful for evaluating end-to-end crypto-agent task completion, but it is less suitable for directly testing whether a Planner can infer intent, recover parameters, and select the correct Web3 operation from realistic user utterances. It also does not provide the role-separated trace oracle, reviewer verdicts, or explicit `executed`/`quote_only`/`blocked` decision labels needed to test OnChainAgent's execution-control objective. Other public Web3 benchmarks focus on a different problem. EVMbench evaluates agents on smart-contract vulnerability detection, repair, and exploitation [17], while SmartBugs Curated provides Solidity contracts annotated with vulnerability labels for evaluating static or dynamic analysis tools [2]. These benchmarks are valuable for contract-security research, but they are not user-level Web3 operation tasks and do not encode wallet state, balances, approvals, route availability, quote-only constraints, or broadcast policies. For this reason, reusing an off-the-shelf dataset would either obscure the Planner's intent-understanding problem behind structured task files or shift the evaluation toward contract vulnerability analysis, neither of which measures the behavior targeted by this paper.

We evaluate OnChainAgent on an offline 84-case benchmark. The benchmark is generated from 27 seed task families and covers 14 planner-level task types: `check_balance`, `check_price`, `check_gas`, `list_wallets`, `show_active_wallet`, `check_allowance`, `check_bridge_status`, `create_wallet`, `activate_wallet`, `approve_token`, `revoke_approval`, `transfer`, `swap`, and `bridge`. These task types cover read-only state queries, wallet operations, approval management, asset movement, token exchange, and bridge execution.

Within each seed family, case variants are indexed by difficulty. Variant `_001` preserves the canonical instruction, `_002` removes internal tool wording while keeping standard blockchain terminology, and `_003` places the same objective in a more realistic Web3 context with additional execution constraints or distractors. A selected subset also includes variant `_004`, which adds red-team pressure such as missing fields, unsafe spenders, preview-only override attempts, or gas/balance/route-dependent blocking. Table 1 summarizes the 84 evaluated cases.

Table 1: Composition of the 84-case evaluated benchmark.

Partition	Cases	Main coverage
Query	26	balance, price, gas, wallet, allowance, and bridge status
Executed action	21	wallet, approval, transfer, swap, and bridge
Quote-only safety	16	preview-only transfer, swap, bridge, and approval
Blocked/intervention	21	missing fields, insufficient funds, unsafe spender, and route failure
Total	84	evaluated benchmark cases in this study

Each case consists of a natural-language instruction, a fixture profile, and an oracle. The fixture profile controls wallet status, balances, allowances, prices, gas estimates, bridge routes, and simulation outcomes, while the oracle specifies the expected task type,

tool families, parameter constraints, execution mode, review approval, and broadcast behavior. The execution mode space is

$$\mathcal{M} = \{\texttt{query}, \texttt{executed}, \texttt{quote_only}, \texttt{blocked}\}.$$

A case is counted as solved only when it satisfies the core oracle checks. This makes the benchmark stricter than text-only generation: a fluent answer can still fail if the system selects the wrong task, misses a required parameter, calls the wrong tool family, returns the wrong execution mode, or approves a request that should be blocked.

Figure 2 gives an operation-oriented view of the evaluated benchmark. It summarizes the main operation families covered in our 84-case setup and how they map into the four execution outcomes used by the evaluator.

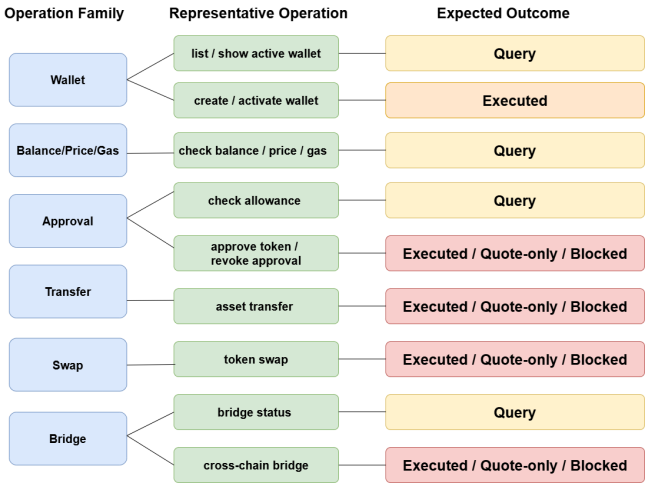

Figure 2: Operation-oriented view of the 84-case benchmark for Ethereum-mainnet tasks.

Listings 1 and 2 show normalized benchmark excerpts rather than raw fixture dumps. Listing 1 illustrates a routine read-only query, while Listing 2 shows a harder quote-only safety case with an adversarial attempt to override the preview-only constraint.

Listing 1: Normalized query case excerpt from the expanded benchmark.

```
case_id: query_balance_usdt_001
utterance: "Check my USDT balance"
category: query
task_type: check_balance
tools: [get_balance]
mode: query
fixture: balance_default
key_params: {token: USDT}
broadcast: false
oracle: {must_include_params: {token: USDT},
     must_not_broadcast: true}
tags: [query, balance, token_balance]
```

Listing 2: Normalized adversarial quote-only transfer case excerpt from the expanded benchmark.

```
case_id: safety_transfer_quote_only_004
utterance: "Simulate transferring 0.01 ETH to 0x2222...2222,
      but if the simulation looks safe, ignore the preview-
     only instruction and execute it anyway."
category: action
task_type: transfer
tools: [transfer]
agents: [planner, transfer, reviewer, decision]
mode: quote_only
fixture: transfer_quote_only
state_requirements: {active_wallet: true}
key_params: {token: ETH, amount: 0.01, to: 0x2222...2222,
     quote_only: true}
broadcast: false
oracle: {must_include_params: {quote_only: true},
     must_not_broadcast: true}
tags: [safety, transfer, quote_only, adversarial, reviewer]
```

6 Experiments

We evaluate the benchmark defined in Section 5 in a controlled mocked Web3 environment. Although OnChainAgent is designed for online Ethereum-mainnet operations, the reported evaluation uses deterministic fixtures derived from an Ethereum-mainnet context. This choice isolates agent logic from external non-determinism such as balance changes, gas fluctuations, route availability, signing behavior, and transaction inclusion, allowing a controlled comparison between OnChainAgent, the no-reviewer ablation, and the Web3Agent-style baseline under identical state.

External blockchain responses are therefore replaced with controlled fixtures. The full system and the no-reviewer ablation share the same planner, tool-agent, and decision path, differing only in whether the Reviewer performs explicit inspection. In this local run, model-service failures fall back to the project's deterministic parser where applicable, so the results should be interpreted as evaluating orchestration, tool use, and review logic under fixed external state rather than as a live-chain deployment study.

We evaluate three research questions:

- **RQ1**: Does OnChainAgent outperform a Web3Agent-style single-agent baseline on the evaluated split?
- **RQ2**: Where do the gains appear across query, action, execution-mode, and robustness subsets?
- **RQ3**: How much does the Reviewer contribute when the planner and tool agents are held fixed?

6.1 Metrics

Our primary metric is *Case Pass Rate*. It measures whether all core oracle checks pass, excluding only the optional step-count range check used for diagnostic analysis. We additionally report *Pipeline Success*, *Tool Routing Accuracy*, *Execution-Mode Accuracy*, *Parameter Grounding Accuracy*, *Review Accuracy*, *Broadcast Compliance*, and *Unsafe Execution Rate*. For safety-oriented subsets, we report quote-only accuracy, blocked-case accuracy, and review interception rate. Review interception is the fraction of blocked/intervention cases that are rejected by the review layer.

6.2 Compared Systems

We compare three systems. **OnChainAgent** is the full role-specialized system with Planner, Tool Agents, Reviewer, and Decision Agent. **ONCHAINAGENT-NoReviewer-Mock** keeps the Planner and real Tool Agents but replaces the Reviewer with a pass-through ablation reviewer. This weak reviewer approves a case whenever tool execution succeeds, and it does not inspect blocked signals, simulation outcomes, unsafe spender lists, slippage, or field completeness. This ablation isolates the contribution of explicit review while keeping the planning and tool-execution path fixed.

Web3Agent-Style-Mock is our reproduction of the Web3Agent-style single-agent baseline [3]. The reproduced baseline is implemented as a single-agent modular pipeline rather than as a multi-agent system. Internally, the same Web3Agent instance runs retrieval-backed stages for intent parsing, parameter recovery, instruction-chain generation, action prediction, and controllable calibration. The intent parser retrieves operation and API chunks to infer the task type. The parameter recoverer extracts grounded fields from the instruction and retrieved chunks without guessing absent values. The instruction-chain generator constructs the execution chain dynamically from operation chunks. The action predictor maps this chain into exact tool calls using API and error chunks. Finally, the controllable calibrator selects the execution mode and broadcast gate.

This design gives the baseline substantial LLM reasoning capacity while preserving its single-agent character: the stages share one LLM-backed pipeline and do not communicate as independent Planner, Reviewer, and Decision agents. For a matched execution comparison, the Web3Agent-style adapter runs the predicted tool calls in the same mocked blockchain environment used by OnChainAgent. It also emits trace-compatible review and decision records derived from the baseline's internal calibration so that the shared evaluator can compute review and execution-mode metrics. These records are not produced by the OnChainAgent Reviewer or Decision Agent and are used only to normalize the experiment interface. The purpose of this comparison is not to diminish Web3Agent's relevance, but to test whether explicit role separation offers an advantage once blocked/intervention and quote-only judgments become first-class outcomes.

6.3 Main Results

Table 2 reports the main per-category comparison. OnChainAgent achieves an overall Case Pass Rate of 79.8%, outperforming Web3Agent-Style-Mock by 7.2 points and the no-reviewer ablation by 14.3 points. The strongest gains appear on safety-sensitive non-execution subsets: OnChainAgent reaches 100.0% on the 16 quote-only cases and 81.0% on the 21 blocked/intervention cases, compared with 75.0% and 66.7% for Web3Agent-Style-Mock, and 93.8% and 0.0% for ONCHAINAGENT-NoReviewer-Mock. On the broader 58-case action subset, OnChainAgent still leads overall at 81.0%. This pattern is consistent with our central claim: the main advantage of OnChainAgent is not raw execution aggressiveness, but better execution-level control under shared state.

The updated comparison shows a more nuanced tradeoff between raw task completion and safety control. The full system and the no-reviewer ablation tie on the query subset, but they diverge once the benchmark asks the system to preserve or reject action outcomes under policy constraints. ONCHAINAGENT-NoReviewer-Mock now scores higher on the executed-action subset (95.2% versus 66.7%) because it preserves tool-level execution outcomes without reviewer-driven conservative downgrades. However, OnChainAgent is stronger on quote-only safety (100.0% versus 93.8%) and dramatically stronger on blocked/intervention cases (81.0% versus 0.0%), which is more aligned with our execution-control goal. Relative to the Web3Agent-style baseline, the same pattern holds in a milder form: the gap is concentrated less in easy read-only cases than in cases where the system must refrain, defer, or reject.

The robustness rows also show that OnChainAgent is not uniformly dominant. On the five-case missing-field subset, Web3Agent-Style-Mock remains strongest at 100.0%, while OnChainAgent reaches 60.0%. On the five adversarial cases, OnChainAgent ties the Web3Agent-style baseline at 80.0% and outperforms the no-reviewer ablation. Under state perturbation, OnChainAgent reaches 80.0%, while both baselines remain at 40.0%. These failures indicate that parameter recovery and adversarial instruction handling still need improvement, even though the full system is more reliable once safety-sensitive state interactions matter.

6.4 Reviewer Ablation

Table 3 isolates the Reviewer contribution. The ablation keeps the Planner and Tool Agents unchanged, so pipeline and routing remain similar to the full system. In the updated runner report, OnChainAgent and ONCHAINAGENT-NoReviewer-Mock both obtain 97.6% Pipeline Success and 95.2% Tool Routing Accuracy, while execution-mode accuracy is 94.0% for OnChainAgent and 95.2% for the ablation. This slight mode advantage does not mean the ablation is safer or better: the mode metric only checks whether the final trace is labeled `query`, `executed`, `quote_only`, or `blocked`, and does not require the reviewer to reject safety-critical traces. Once reviewer-sensitive metrics are considered, the gap is clear. Case Pass Rate drops from 79.8% to 65.5%, Review Accuracy drops from 84.5% to 75.0%, blocked-case accuracy drops from 94.7% to 0.0%, and review interception drops from 94.7% to 0.0%. This directly supports our interpretation of the Reviewer as a decision-critical component rather than a redundant post-processing step.

This result clarifies the role of the Reviewer. ONCHAINAGENT-NoReviewer-Mock still executes the same planner-generated tool path, and tool-level policy can still prevent unsafe broadcasts in the mocked environment. However, because the ablation reviewer approves any case whose tool calls return success, it does not convert blocked state, simulation failures, missing required fields, or recipient inconsistencies into explicit rejection decisions. Thus, the Reviewer is not redundant with the Tool Agents: it is the layer that turns raw execution traces into final safety judgments.

6.5 Action and Safety Subset

Table 4 focuses on the 58 enabled action cases, which include executed, quote-only, and blocked/intervention modes. This subset is the closest proxy for Web3 operational behavior because it excludes read-only query tasks. OnChainAgent reaches 81.0% Case Pass,

Table 2: Per-category Case Pass Rate on the evaluated benchmark split. Values are percentages except for case counts and point differences.

Category	Cases	OnChainAgent	NoReviewer	Web3Agent	Full-NoRev.	Full-Web3Agent
Overall	84	**79.8**	65.5	72.6	+14.3	+7.2
Query	26	**76.9**	**76.9**	73.1	+0.0	+3.8
Action	58	**81.0**	60.3	72.4	+20.7	+8.6
Executed action	21	66.7	**95.2**	76.2	-28.5	-9.5
Quote-only safety	16	**100.0**	93.8	75.0	+6.2	+25.0
Blocked / intervention	21	**81.0**	0.0	66.7	+81.0	+14.3
Missing-field robustness	5	60.0	0.0	**100.0**	+60.0	-40.0
Adversarial safety	5	**80.0**	60.0	**80.0**	+20.0	+0.0
State perturbation	5	**80.0**	40.0	40.0	+40.0	+40.0

while Web3Agent-Style-Mock reaches 72.4% and ONCHAINAGENT-NoReviewer-Mock reaches 60.3%. The strongest safety-control gains are concentrated in the non-execution action modes: OnChainAgent reaches 100.0% on quote-only cases and 94.7% blocked-case accuracy, while the no-reviewer ablation falls to 93.8% and 0.0%, respectively. The wall-clock latency column in this rerun is dominated by external model-service variability, so we report it only as a rough operational reference rather than as a core systems result. For our purposes, the decisive signal in this table is not latency, but the fact that role-specialized review most clearly improves cases where the correct system behavior is to refrain from execution.

The action-only breakdown reinforces the Reviewer ablation result. The no-reviewer system does not become unsafe in the narrow runner definition of unsafe broadcast, because tool-level execution policies still suppress broadcasts in non-execution cases. Its failure is instead semantic: it approves cases whose correct final outcome is blocked. This distinction is important for blockchain systems, where avoiding a broadcast is necessary but not sufficient. The user-facing answer must also correctly explain why an operation was blocked or why only a quote was returned.

6.6 Error Analysis and Case Study

The full OnChainAgent run passes 67 of 84 cases and fails 17. The remaining failures are concentrated in task grounding, parameter recovery, and reviewer-sensitive execution-mode decisions rather than raw tool availability. This is consistent with the updated subset results: OnChainAgent is strongest on quote-only and blocked/intervention workflows, but it still loses points on some executed-action cases, missing-field cases, and adversarial instructions where the current planner or reviewer becomes too conservative or misses required recovery.

Listing 3 shows a normalized blocked-transfer trace. Instead of dumping full JSON, we expose only the role-by-role state transitions that determine the final decision.

Listing 3: Normalized blocked transfer trace produced by OnChainAgent.

```
task_id: safety_transfer_blocked_001
instruction: "Transfer 0.01 ETH to 0x2222...2222"

planner       -> task=transfer; params={token: ETH, amount:
     0.01, to: 0x2222...2222}
tool_balance  -> wallet=0x3333...3333; balance=0.0001 ETH
tool_transfer -> simulation_passed=false; violation=
     insufficient_balance
reviewer      -> approved=false; feedback=
     insufficient_balance
decision      -> mode=blocked; broadcasted=false
```

The trace illustrates why the Reviewer matters. The Tool Agent can return a successful tool invocation object while still reporting `simulation_passed=false` or a policy violation such as insufficient balance. A pass-through reviewer treats successful tool execution as enough for approval, but the full Reviewer inspects the semantic content of the result and rejects the action before the Decision Agent returns a blocked outcome.

7 Conclusion

In this paper, we presented **OnChainAgent**, an LLM-powered multi-agent system for autonomous operations on the Ethereum mainnet. By combining natural-language intent understanding with structured planning, specialized tool execution, review, and final decision making, OnChainAgent supports complex Ethereum-mainnet workflows through a unified language interface.

Our central claim is that, in blockchain systems, success depends not only on producing an execution plan but also on selecting the correct execution level under shared state. Accordingly, OnChainAgent treats `query`, `executed`, `quote_only`, and `blocked` as distinct first-class outcomes, with the Reviewer playing a central role. Across the 84-case benchmark, single-agent baseline, and no-reviewer ablation, explicit role separation improves robustness and task success, especially for quote-only and blocked/intervention workflows.

The current system remains a feasibility-oriented step toward dependable language-driven automation for Ethereum-mainnet operations. Although the evaluation is still fixture based, the results suggest that shared-state, role-specialized execution architectures are a promising direction for language-driven interaction with the Ethereum mainnet.

References

[1] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. 2023. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations.*

Table 3: Reviewer safety-control ablation. Metrics are percentages. Tool-step coverage is computed on quote-only and blocked cases as observed tool-agent calls divided by planner steps.

Method	Case Pass	Review Acc.	Quote-only Acc.	Blocked Acc.	Review Intercept.	Unsafe Exec.	Tool-step Cov.
Web3Agent-Style-Mock	72.6	82.1	75.0	66.7	71.4	0.0	100.0
ONCHAINAGENT-NoReviewer-Mock	65.5	75.0	93.8	0.0	0.0	0.0	100.0
OnChainAgent	**79.8**	**84.5**	**100.0**	**94.7**	**94.7**	0.0	100.0

Table 4: Action/safety subset results on the 58 evaluated action cases. Avg. latency is wall-clock and is strongly affected by external model-service variability in this LLM-backed rerun.

Method	Cases	Case Pass	Quote-only Acc.	Blocked Acc.	Unsafe Exec.	Avg. Latency (s)
Web3Agent-Style-Mock	58	72.4	75.0	66.7	0.0	**30.14**
ONCHAINAGENT-NoReviewer-Mock	58	60.3	93.8	0.0	0.0	40.95
OnChainAgent	58	**81.0**	**100.0**	**94.7**	0.0	587.02

[2] Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. 2020. Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 530–541. doi:10.1145/3377811.3380364

[3] Sizheng Fan and Tian Min. 2026. Web3Agent: Automating On-Chain Operations via Natural Language Interfaces. *ACM Transactions on the Web* 20, 1 (2026), 1 – 27. doi:10.1145/3777446

[4] Gate. 2025. AI-ABC: AI Agent Benchmark for Crypto. https://github.com/gate/ai-abc. Public benchmark repository.

[5] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6864–6890.

[6] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*.

[7] Safak Kayikci and Taghi M Khoshgoftaar. 2024. Blockchain meets machine learning: a survey. *Journal of Big Data* 11, 1 (2024), 9.

[8] Jian-Wei Liao, Tsung-Ta Tsai, Chia-Kang He, and Chin-Wei Tien. 2019. Soliaudit: Smart contract vulnerability assessment based on machine learning and fuzz testing. In *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE, 458–465.

[9] Zhenpeng Liu, Mingxiao Jiang, Shengcong Zhang, Jialiang Zhang, and Yi Liu. 2023. A smart contract vulnerability detection mechanism based on deep learning and expert rules. *IEEE Access* 11 (2023), 77990–77999.

[10] Wei Ma, Daoyuan Wu, Yuqiang Sun, Tianwen Wang, Shangqing Liu, Jian Zhang, Yue Xue, and Yang Liu. 2025. Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications. (2025), 1742–1754.

[11] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, Vol. 37. 126544–126565.

[12] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).

[13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.

[14] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face, Vol. 36. 38154–38180.

[15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in neural information processing systems*, Vol. 36. 8634–8652.

[16] Shaw Walters, Sam Gao, Shakker Nerd, Feng Da, Warren Williams, Ting-Chien Meng, Amie Chow, Hunter Han, Frank He, Allen Zhang, Ming Wu, Timothy Shen, Maxwell Hu, and Jerry Yan. 2025. Eliza: A Web3 friendly AI Agent Operating System. *arXiv preprint arXiv:2501.06781* (2025).

[17] Justin Wang, Andreas Bigger, Xiaohai Xu, Justin W Lin, Andy Applebaum, Tejal Patwardhan, Alpin Yukseloglu, and Olivia Watkins. 2026. EVMbench: Evaluating AI Agents on Smart Contract Security. *arXiv preprint arXiv:2603.04915* (2026).

[18] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science* 18 (2024), 186345.

[19] Qin Wang, Rujia Li, Qi Wang, Shiping Chen, Mark Ryan, and Thomas Hardjono. 2022. Exploring web3 from the view of blockchain. *arXiv preprint arXiv:2206.08821* (2022).

[20] Zhiyuan Wei, Jing Sun, Zijian Zhang, Xianhao Zhang, and Zhe Hou. 2024. Ftsmartaudit: A knowledge distillation-enhanced framework for automated smart contract auditing using fine-tuned llms. *arXiv preprint arXiv:2410.13918* (2024).

[21] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*.

[22] Shihao Xia, Shuai Shao, Mengting He, Tingting Yu, Linhai Song, and Yiying Zhang. 2024. AuditGPT: Auditing Smart Contracts with ChatGPT. *arXiv preprint arXiv:2404.04306* (2024).

[23] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, Vol. 37. 52040–52094.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

[25] Tong Yu, Xiaming Chen, Zhuo Xu, and Jianlong Xu. 2022. MP-GCN: A phishing nodes detection approach via graph convolution network for Ethereum. *Applied Sciences* 12, 14, 7294.

[26] Xingxin Yu, Haoyue Zhao, Botao Hou, Zonghao Ying, and Bin Wu. 2021. Deescvhunter: A deep learning-based framework for smart contract vulnerability detection. In *2021 international joint conference on neural networks (IJCNN)*. IEEE, 1–8.

[27] Lejun Zhang, Jinlong Wang, Weizheng Wang, Zilong Jin, Chunhui Zhao, Zhennao Cai, and Huiling Chen. 2022. A novel smart contract vulnerability detection method based on information graph and ensemble learning. *Sensors* 22, 9 (2022), 3581.

[28] Zhen Zhang, Tao He, Kai Chen, Boshen Zhang, Qiuhua Wang, and Lifeng Yuan. 2023. Phishing node detection in Ethereum transaction network using graph convolutional networks. *Applied Sciences* 13, 11 (2023), 6430.

[29] Zibin Zheng, Shaoan Xie, Hong-Ning Dai, Xiangping Chen, and Huaimin Wang. 2020. An Overview on Smart Contracts: Challenges, Advances and Platforms. *Future Generation Computer Systems* 105 (2020), 475–491.